

God Tier Task

Build a Distributed Job Queue: From Scratch

The Brief

You are going to build a production-grade job queue entirely from scratch. No BullMQ. No Celery. No SQS. No Sidekiq. Just raw code, a PostgreSQL database, and the real world fighting back.

This task is deceptively simple on the surface. A table and a loop, how hard can it be? You'll find out. Every bug you hit, every race condition you chase, every mystery you solve will teach you something that no tutorial ever could. By the end, when you read the Kafka paper or the SQS docs, you won't be reading theory. You'll be recognising solutions to problems you personally bled over.

Rules

- No job queue libraries. Build every component yourself.
- Language is your choice. PostgreSQL is the required datastore.
- Each phase must be working and code-reviewed before moving to the next.
- Write a short technical write-up after each phase explaining what you built and what surprised you.
- The chaos test in Phase 5 must pass with zero job loss. No exceptions.

The Five Phases

Phase 1 The Naive Version

1	Design a jobs table in PostgreSQL with status, payload, created_at, attempts
2	Write a worker process that polls for pending jobs and executes them
3	Implement basic status transitions: pending → running → completed / failed
4	Add a simple REST API to enqueue jobs and check their status

Challenge: *You'll get this working in a day. Then run two workers simultaneously and watch chaos unfold.*

Phase 2 Make It Not Break

1	Implement automatic retries with exponential backoff
2	Add a dead-letter queue for jobs that exceed max retry attempts
3	Implement idempotency keys so re-enqueuing the same job is safe
4	Research and understand the difference between at-least-once and exactly-once delivery — document your findings

Challenge: *A job that charges a customer runs twice. Figure out why, then make it impossible.*

Phase 3 Scale It Out

1	Fix the dual-worker race condition using SELECT FOR UPDATE SKIP LOCKED
2	Implement worker heartbeats — each worker pings a heartbeats table every N seconds
3	Write a watchdog process: detect dead workers and re-queue their in-progress jobs
4	Benchmark: how many jobs/sec can your queue process with 1, 4, and 8 workers?

Challenge: *Kill a worker mid-job with SIGKILL. The job must recover automatically within 30 seconds.*

Phase 4 Observe It

1	Expose a /metrics endpoint with: queue depth, jobs/sec, p50/p95/p99 latency, error rate
2	Add structured logging (JSON) to every state transition with job_id, worker_id, duration
3	Build a minimal dashboard (even a terminal UI is fine) showing live queue health
4	Set up an alert: notify if queue depth exceeds 1,000 unprocessed jobs

Challenge: *With no visibility into the queue, you are flying blind. Observability is not optional.*

Phase 5 Break It On Purpose

1	Write a chaos script that kills random workers every 30 seconds
2	Inject artificial DB latency (<code>pg_sleep</code>) and observe queue behavior under stress
3	Flood the queue with 100,000 jobs at once — measure throughput, identify bottlenecks
4	Write a post-mortem: what broke, why, and how you fixed it

Challenge: *If it only works under ideal conditions, it doesn't work. Production is chaos.*

What You Will Actually Learn

This isn't a list of technologies. These are real concepts, learned the hard way.

Concept	Why it matters
Transactions & locking	<code>SELECT FOR UPDATE</code> , <code>SKIP LOCKED</code> , isolation levels
Distributed state	Coordinating multiple workers without a central coordinator
Failure modes	Network partitions, process crashes, half-applied writes
Idempotency	Why making operations safe to repeat is one of the hardest problems
Observability	Metrics, structured logging, percentile latencies
Backpressure	What happens when producers outpace consumers
Graceful shutdown	Draining in-flight work before a process exits
Chaos engineering	Deliberately breaking things to find what you haven't thought of

Final Deliverable

A GitHub repository containing all five phases with:

- Clean, readable code with a proper README
- Docker Compose setup so anyone can run the full system locally
- A technical write-up for each phase (can be markdown)
- Phase 5 chaos test script with documented results
- Benchmarks: throughput numbers at 1, 4, and 8 workers

When you finish this, you will understand distributed systems at a level most engineers never reach. Good luck.